



Quantum Bluff

FINAL PROJECT REPORT

University of Strasbourg

Bachelor of Science in Computer Science (L3)

Project manager: Omar

Team members: Linda, Massi, Abdoulaye, Soheil, Mohamed, Azra, Yigit

Academic year 2025–2026

Table of contents

1	Introduction	1
1.1	Context and Objectives	1
1.2	Scope of this document	1
2	Project Evolution	2
2.1	Initial specifications (poker-first)	2
2.2	Scope expansion (casino, tournaments, social, progression)	2
2.3	Why the narrative moved from “catalog” to “appendices”	2
3	Project Management	3
3.1	Initial V-model phase	3
3.2	Transition to an agile-inspired workflow (Jira, Scrum practices)	3
3.3	Planning, coordination, and reporting	3
3.4	Product management mechanics	3
4	Development Tools and Technologies	4
4.1	Frontend and backend	4
4.2	Database and infrastructure	4
4.3	Version control and collaboration	4
4.4	Testing and deployment	4
5	Use of Artificial Intelligence	5
5.1	In-product AI: poker bots	5
5.2	AI-assisted engineering (tooling)	5
5.3	Limits and verification	5
6	Platform Functionalities	6
7	System Architecture	8
7.1	Monorepo layout	8
7.2	Real-time communication	8
7.3	Persistence and integrity	8
7.4	Diagram	8
8	Security, Testing and Stability	9
8.1	Authentication and abuse controls	9
8.2	Testing and debugging	9
8.3	Performance and stability	9
9	Technical Challenges	10

10 Individual Contributions	11
10.1 Omar – project management and architecture alignment	11
10.2 Other contributors	11
11 Limitations and Future Work	12
11.1 Current limitations	12
11.2 Future improvements	12
12 Conclusion	13
A Appendix A: Repository map and routing	13
B Appendix B: Poker engine and cash game	13
C Appendix C: Bot AI and practice integration	13
D Appendix D: Probabilities, Quantum HUD, and Monte Carlo helper	13
E Appendix E: Hidden betting system	14
F Appendix F: Tournaments and winner side bets	14
G Appendix G: Blackjack solo and multiplayer	14
H Appendix H: Slot machine, roulette, and casino wallet	14
I Appendix I: Social graph, invitations, and peer loans	14
J Appendix J: Progression, challenges, and leaderboards	15
K Appendix K: Authentication, profiles, and i18n	15
L Appendix L: Operations, Redis, CI, and packaging	15
M Appendix M: External and large internal references	15

1 Introduction

1.1 Context and Objectives

Quantum Bluff was developed as part of the final third-year Bachelor’s degree project (L3 Computer Science, French academic system) at the University of Strasbourg. The objective was to deliver a **real-time multiplayer gaming platform** with a professional-grade engineering posture: modular architecture, automated testing, observability hooks, security hardening, and collaborative delivery under time constraints.

The **initial product intent** (before scope expansion) was a Texas Hold’em poker platform: practice mode against AI with multiple difficulty levels, multiplayer cash tables (public/private), configurable blinds, minimum buy-in rules, a probability-oriented HUD, and a **hidden side-bet** subsystem. The technical goals included authoritative server-side rules, transactional chip movements, WebSocket-scale event delivery, and operational readiness (health checks, metrics, recovery paths).

Security and reliability were first-class constraints from the outset: JWT authentication, optional TOTP 2FA, anti-cheat and rate-limiting layers, idempotent handling for sensitive mutations, and schema validation on critical API paths.

1.2 Scope of this document

This report summarizes **management evolution, tooling, delivered capabilities, and architecture**. Exhaustive technical narratives (bots, probabilities, tournaments, hidden bets, etc.) are deferred to **Appendices A–M** to avoid repetition. Repository-relative file paths use `\path{...}` notation.

2 Project Evolution

2.1 Initial specifications (poker-first)

The early scope targeted: Texas Hold'em cash games; **AI practice** with four difficulty tiers (easy, medium, hard, expert) enforced in `server/src/routes/game.api.routes.ts` and `server/src/routes/bot.routes.ts`; public/private multiplayer rooms with SB/BB configuration and minimum balance rules; **real-time** updates via Socket.IO; a **Quantum HUD** for win-equity and hand-category feedback (`client/src/components/QuantumHUD.tsx`); and **hidden bets** resolved server-side (`server/src/poker/hiddenBets/`).

2.2 Scope expansion (casino, tournaments, social, progression)

The delivered system adds: **Blackjack** solo and **multi-table** lobbies (`client/src/pages/Blackjack.tsx`, `client/src/pages/BlackjackMultiLobby.tsx`, `client/src/pages/BlackjackMultiTable.tsx`); **slot** and **roulette** flows reachable from the mini-games hub (`client/src/pages/MiniGames.tsx`); **tournaments** with waiting room, in-tournament table, results, and **winner prediction side bets** (`client/src/features/tournament/`); **leaderboards**; **XP**, badges, daily challenges, daily login streak; **friends**, invitations, join requests, **peer loans** with interest/repayment hooks; **profile** editing with server-stored avatars; **2FA TOTP**; **feedback** and **player reports**; **wallet ledger**, gift codes, and dev recharge helpers; **i18n** in five locales (`client/src/i18n/locales/`); **tutorial** flow (`client/src/pages/TutorialGame.tsx`, `client/src/features/tutorial/`); **admin web console** and packaged targets (**Electron**, **Capacitor**) per `client/package.json`.

2.3 Why the narrative moved from “catalog” to “appendices”

The codebase grew across **client**, **server**, **database**, and **docs/** technical notes. Listing every subsystem in prose would duplicate `docs/ARCHITECTURE.md` and the structured logic notes under `Docs/detail_logique/`. The body therefore stays outcome-oriented; appendices map each major subsystem to code anchors.

3 Project Management

3.1 Initial V-model phase

The project began with a **V-model style** framing: business requirement gathering with end users in mind, benchmarking comparable products, brainstorming sessions **validated collectively** by the team, and a written requirements package plus an initial Gantt chart. A **technical lead** was appointed to structure the MOE (implementation) track while the project manager covered the MOA (needs, prioritization, specification) track until the specification baseline stabilized.

3.2 Transition to an agile-inspired workflow (Jira, Scrum practices)

After an intensive early phase (approximately **two months** of V-model-oriented execution on a **four-month** calendar), the team concluded that strict phase separation slowed integration of new gameplay and infrastructure ideas. The organization shifted toward an **iterative, Scrum-inspired workflow**: **Jira** for epics, user stories, and time-boxed sprints; backlog ordering by **value vs. risk**; twice-weekly synchronization meetings for decisions, unblockers, and demo-ready increments. The project manager studied agile practice material (including *Scrum Life*) to align rituals with the team's reality rather than copying a textbook process wholesale.

3.3 Planning, coordination, and reporting

Coordination relied on: recurring meetings (~twice per week), shared technical documentation under Docs/, internal validation sessions, and continuous integration feedback via `.gitlab-ci.yml` at the repository root. Work was distributed dynamically across frontend, backend, and infrastructure tasks because multiplayer features require tight coupling between UI state, socket events, and persistence.

3.4 Product management mechanics

Backlog prioritization balanced user-visible value (new game modes, social features) against enabling work (Redis adapter, rate limits, recovery jobs, observability). Cross-cutting concerns (security, idempotency, anti-cheat) were escalated early to avoid late integration risk.

4 Development Tools and Technologies

4.1 Frontend and backend

Client: React 19, TypeScript, Vite 7, Tailwind CSS, Radix UI primitives (`client/src/components/ui/`), react-router-dom v7, Redux Toolkit with RTK Query (`client/src/services/api.ts`), socket.io-client (`client/src/services/socket.ts`), i18next (`client/src/i18n/config.ts`).

Server: Node.js (ESM), Express 4, Socket.IO 4, Prisma 6 with PostgreSQL (`server/prisma/schema.prisma`), Redis clients for rate limiting, action logging, and the Socket.IO Redis adapter (`server/src/index.ts`), Zod validation, JWT auth, bcrypt password hashing, sanitize-html where applicable.

4.2 Database and infrastructure

Local PostgreSQL via `database/docker-compose.yml`. Redis for multi-instance coordination and abuse controls. Optional OpenTelemetry and Prometheus-oriented metrics endpoints (`server/src/observability/`).

4.3 Version control and collaboration

Git repository with GitLab CI configuration at `.gitlab-ci.yml`. Documentation and logic notes live under `Docs/` (including `docs/ARCHITECTURE.md` and `Docs/detail_logique/`).

4.4 Testing and deployment

Client: Vitest, Playwright (`client/e2e/`). Server: Jest and Supertest (`server/src/__tests__/_/`). Desktop/mobile packaging: Electron and Capacitor scripts in `client/package.json`.

5 Use of Artificial Intelligence

5.1 In-product AI: poker bots

The shipped product includes **heuristic poker bots** with configurable difficulty. Core decision logic lives in `server/src/logic/botAI.ts`, orchestrated HTTP services in `server/src/services/botAi.service.ts`, and practice-mode integration (`server/src/poker/services/practiceBotTurns.service.ts`). Difficulty is part of the validated API contract (`server/src/routes/bot.routes.ts`).

5.2 AI-assisted engineering (tooling)

Team members used modern **coding assistants** (e.g., IDE-integrated LLMs) for drafting, refactors, and documentation. Generated output was treated as **untrusted**: reviewed in merge requests, validated by automated tests, and rejected when it contradicted server authority or security invariants.

5.3 Limits and verification

Bots are **not** opaque learned models in production: they are inspectable code paths. Assistant-generated snippets were never merged without human review and test signal.

6 Platform Functionalities

Table 1 lists major domains. Details appear only once, in the referenced appendix, to prevent duplication.

Table 1: High-level functional inventory (representative surfaces).

Domain	User-visible capability	Primary UI entry	Deep dive
Poker cash	Multiplayer tables, pot logic, chat, actions	<code>/game,</code> <code>/waiting-room</code>	Appendix B
Poker bots	Practice tables, four difficulties	<code>/bot-configuration</code>	Appendix C
HUD	Win chance and hand category estimates	In-game HUD	Appendix D
Hidden bets	Side markets, pricing, settlement	In-game panel, <code>/results</code>	Appendix E
Blackjack	Solo + multi lobby/table	<code>/blackjack,</code> <code>/blackjack/</code> lobby, <code>/blackjack/</code> <code>table/:gameId</code>	Appendix G
Casino	Slot and roulette	<code>/minigames</code>	Appendix H
Tournaments	Waiting, play, results, winner side bets	<code>/tournaments/..</code> .	Appendix F
Social	Friends, invites, loans	<code>/friends,</code> lobby flows	Appendix I
Progression	XP, badges, daily challenges, login streak, leaderboard	<code>/profile,</code> <code>/leaderboard</code>	Appendix J
Account	Auth, 2FA, avatars, profile edit	<code>/auth,</code> <code>/edit-profile</code>	Appendix K
Admin	Operator web console (JWT admin role)	<code>/admin/console</code>	Appendix L
Ops	Metrics, health, CI, packaging	N/A (infra)	Appendix L
Other UX	Gift codes, free recharge, post-game rating modal	<code>client/src/</code> <code>components/</code> <code>Layout.tsx,</code> lobby	Appendix L
Feedback	User feedback and player reports	Lobby/ <code>settings</code> flows	Appendix L

Domain	User-visible capability	Primary UI entry	Deep dive
Tournament UX	ZipRush mini-game while waiting	<code>client/src/features/tournament/components/ZipRushMiniGame.tsx</code>	Appendix F
Demo	Example and deal teaching routes	<code>/game-example, /game-deal</code>	Appendix B
A11y	Accessibility, table theming, ambient audio	<code>client/src/contexts/AccessibilityContext.tsx, TableThemeContext.tsx, MusicContext.tsx</code>	Appendix A

7 System Architecture

7.1 Monorepo layout

The repository splits into `client/` (Vite SPA), `server/` (Express + Socket.IO gateway), and `database/` (local Postgres). Client bootstraps providers in `client/src/main.tsx` and routes in `client/src/App.tsx`. Server composes middlewares and REST mounts in `server/src/index.ts`, then attaches `server/src/sockets/game.gateway.ts` to the HTTP server.

7.2 Real-time communication

Socket.IO uses path `/socket.io`. Authentication mirrors REST JWT validation (`server/src/middleware/socketAuth.middleware.ts`). Redis adapter enables horizontal scale-out when Redis is available.

7.3 Persistence and integrity

Prisma models capture users, games, tournaments, loans, hidden bet tickets, wallet ledger rows, and more (see `server/prisma/schema.prisma`). Critical monetary flows favor server-side orchestration and short transactions.

7.4 Diagram

Figure 1 summarizes the request path. For a publication-quality diagram, export a Mermaid figure from `docs/ARCHITECTURE.md` to PNG and include it with `\includegraphics`.

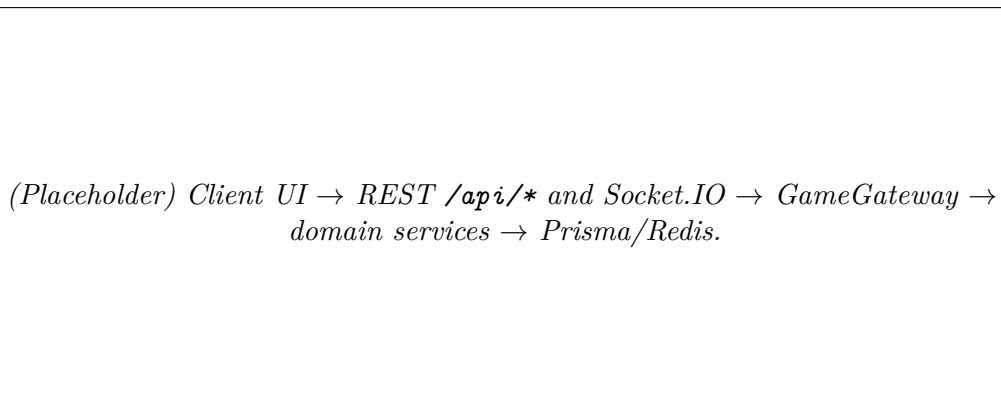


Figure 1: High-level runtime flow (placeholder box; replace with exported diagram if desired).

8 Security, Testing and Stability

8.1 Authentication and abuse controls

JWT on HTTP and sockets; optional TOTP 2FA (`server/src/routes/twofa.routes.ts`); Helmet and CORS allowlists (`server/src/index.ts`); segmented rate limits for bot, slot, roulette, blackjack, hidden-bets, and multi-blackjack routes; anti-cheat middleware and gateway-side monitors (`server/src/services/antiCheat.service.ts`, `server/src/sockets/game.gateway.ts`); idempotency middleware for sensitive mutations (`server/src/middleware/idempotency.middleware.ts`).

8.2 Testing and debugging

Broad automated coverage on server and client; Playwright smoke tests. Additional manual scenarios are documented under `Docs/POKER_SCENARIOS.md` and related files.

8.3 Performance and stability

Redis-backed Socket.IO adapter, structured logging, Prometheus metrics endpoint, readiness endpoints, graceful shutdown hooks (`server/src/index.ts`).

9 Technical Challenges

Concurrency around shared tables: poker and blackjack runtimes use locking strategies (see `server/src/poker/services/pokerTableLock.service.ts` and blackjack lock services) to prevent split-brain chip mutations.

Recovery after process restarts: blackjack and tournament subsystems implement recovery services invoked at boot (`server/src/index.ts`).

Hidden bet settlement coupling: market resolution must align with poker street transitions; implemented in hidden-bet resolver modules (Appendix E).

Internationalization without clobbering user preference: guest screens (StartScreen, Auth, global loader) force English using `react-i18next` with a fixed `lng: 'en'` override so the global language stored for authenticated use is unchanged (`client/src/pages/Auth.tsx`, `client/src/contexts/LoaderContext.tsx`).

10 Individual Contributions

10.1 Omar – project management and architecture alignment

Organizational narrative: Section 3. Beyond ceremonies and backlog arbitration, Omar coordinated **cross-cutting technical commitments**: early adoption of Redis for Socket.IO horizontal scale-out and rate limiting, alignment between Prisma schema changes and gameplay features, and risk reviews before widening scope (tournaments, casino mini-games, loans). Interface with the appointed technical lead focused on sequencing risky merges (socket gateway, recovery jobs) away from demo deadlines.

10.2 Other contributors

Je ne connais pas, je n'ai pas accès (information non présente dans le dépôt).

The repository does not contain signed-off, per-person effort logs, story point assignments, or validated textual attributions beyond this report's author field. Replace this block with team-approved text if provided.

11 Limitations and Future Work

11.1 Current limitations

- **Documentation volume vs. examiners' time:** internal generated reports (e.g., `Docs/Rapport_final/RAPPORT_TECHNIQUE_COMPLET.md`) are useful as indexes but are not all human-curated line-by-line.
- **Dual loader providers:** `client/src/main.tsx` and `client/src/App.tsx` both wrap `LoaderProvider`; harmless but confusing—should be deduplicated.
- **Packaging surface area:** Electron/Capacitor increase release engineering work (signing, store policies) beyond the core web path.

11.2 Future improvements

Hardening continuous deployment secrets, expanding observability dashboards, deeper bot modeling (still code-first), and richer tournament analytics.

12 Conclusion

Quantum Bluff exceeded its initial poker-only charter by integrating multiple casino modes, tournaments, social lending, progression systems, and operational hardening while keeping a monorepo with clear separation between UI, API, and authoritative game logic. The project demonstrates that a student team can approach industry-like constraints—real-time state, monetary integrity, security, and observability—when management explicitly trades early waterfall rigidity for controlled agile iteration.

A Appendix A: Repository map and routing

Client routes are declared in `client/src/App.tsx` (public: `/`, `/auth`, `/auth/admin`; authenticated: `lobby`, `games`, `tournaments`, `profile`, `friends`, etc.). Global providers stack in `client/src/main.tsx` (Redux, sockets, HUD, hidden bets, audio, toasts, user). Server REST mounts and health/metrics endpoints are wired in `server/src/index.ts`. A narrative overview also exists in `docs/ARCHITECTURE.md`.

B Appendix B: Poker engine and cash game

Authoritative table logic centers on `server/src/logic/GameTable.ts` and `server/src/logic/CashGameController.ts`. Client gameplay UI is concentrated in `client/src/pages/Game.tsx` with reusable table components under `client/src/components/`. See `Docs/detail_logique/backend/01_poker_moteur.md` and `Docs/detail_logique/backend/02_cashgame_controller.md`.

C Appendix C: Bot AI and practice integration

Bot difficulty enums are enforced at route validation. Core heuristics live in `server/src/logic/botAI.ts` with HTTP orchestration in `server/src/services/botAi.service.ts` and practice chaining in `server/src/poker/services/practiceBotTurns.service.ts`. Additional design notes: `Docs/IA_EXPERT_BOT.md`.

D Appendix D: Probabilities, Quantum HUD, and Monte Carlo helper

The HUD consumes derived probabilities and hand categories via `client/src/context/QuantumHUDContext.tsx` and `client/src/components/QuantumHUD.tsx`. Client-side Monte Carlo equity estimation is implemented in `client/src/utils/pokerMonteCarloEquity.ts`. Hand taxonomy and combination positioning for UX are additionally documented in `Docs/final_report/QUANTUM_COMBI.adoc`. These UI estimates are distinct from server-side settlement logic for wagering products.

E Appendix E: Hidden betting system

Hidden bets combine market definitions, ticket lifecycle, and resolver routines under `server/src/poker/hiddenBets/`. HTTP surface: `server/src/routes/hiddenBets.routes.ts`. Deep specifications: `Docs/intern/HIDDEN_BETS_SPEC.md`, `Docs/detail_logique/backend/07_hidden_bets_moteur.md`.

F Appendix F: Tournaments and winner side bets

Tournament services, scheduler, and recovery live under `server/src/tournament/`. Winner-bet services: `server/src/tournament/winnerBets/tournamentWinnerBet.service.ts` with routes composed under `/api/tournaments`. Client feature module: `client/src/features/tournament/`. The waiting-room experience embeds a lightweight skill mini-game (`ZipRushMiniGame.tsx`) to retain engagement between bracket transitions. Reference notes: `Docs/intern/TOURNAMENT_FLOW_REAL_VS_EXPECTED.md`.

G Appendix G: Blackjack solo and multiplayer

Solo blackjack routes: `server/src/routes/blackjack.routes.ts`. Multi-table orchestration: `server/src/routes/blackjackMulti.routes.ts` with state stores and recovery under `server/src/blackjack/`. UI: `client/src/pages/Blackjack.tsx`, lobby/table pages, and `client/src/components/blackjack/`. Notes: `Docs/detail_logique/backend/03_blackjack_solo_moteur.md`, `Docs/detail_logique/backend/04_blackjack_multi_moteur.md`.

H Appendix H: Slot machine, roulette, and casino wallet

Game logic modules: `server/src/logic/slotMachine.ts`, `server/src/logic/roulette.ts`. REST routers: `server/src/routes/slot.routes.ts`, `server/src/routes/roulette.routes.ts`. Wallet ledger services under `server/src/casino/` and `server/src/wallet/`. Extra docs: `Docs/detail_logique/backend/06_slot_moteur.md`, `Docs/detail_logique/backend/05_roulette_moteur.md`, `Docs/final_report/SLOT_MACHINE.adoc`, `Docs/final_report/ROULETTE.adoc`.

I Appendix I: Social graph, invitations, and peer loans

Friends and messaging surfaces: `client/src/pages/Friends.tsx`. REST: `server/src/routes/friends.routes.ts`, game invitations via `server/src/routes/invitation.routes.ts` (also mounted at `/api/invitations`), loan routes mounted with friends (`server/src/routes/friendLoan.routes.ts`). Service layer: `server/src/services/friendLoan.service.ts` with socket emissions `server/src/services/friendLoan.emit.ts`. Note: `Docs/detail_logique/backend/09_friend_loans_moteur.md`.

J Appendix J: Progression, challenges, and leaderboards

Gamification hooks in logic (`server/src/logic/gamification.ts`) and dedicated routes for daily challenges and login streak (`server/src/dailyChallenges/`, `server/src/dailyLogin/`). Leaderboard router: `server/src/routes/leaderboard.routes.ts`. Summary: `Docs/detail_logique/backend/10_gamification_moteur.md`.

K Appendix K: Authentication, profiles, and i18n

Auth REST: `server/src/routes/auth.routes.ts`; TOTP: `server/src/routes/twofa.routes.ts`. Client auth UI: `client/src/pages/Auth.tsx`. Profile editing and avatars: `client/src/pages/EditProfile.tsx` with storage fields on `User` in Prisma. Five translation bundles under `client/src/i18n/locales/`.

L Appendix L: Operations, Redis, CI, and packaging

Redis clients for Socket.IO adapter: `server/src/config/socketIoRedis.ts`. Observability: `server/src/observability/`. CI: `.gitlab-ci.yml`. Desktop/mobile: Capacitor/Electron targets in `client/package.json`. Admin console: `client/src/pages/AdminConsole.tsx` with server routes `server/src/routes/adminConsole.routes.ts`. Dev-only admin tools are gated by `!env.isProduction` in `server/src/index.ts`.

M Appendix M: External and large internal references

- Initial formal requirements PDF: `Docs/Cahier_de_charge_6A_PI.pdf` (not embedded here).
- Auto-generated wide-coverage technical compendium: `Docs/Rapport_final/RAPPORT_TECHNIQUE_COMPLET.md` (use as index; verify claims against code).
- Security dossiers: `Docs/Architecture/SECURITE_A_TO_Z.md`, `Docs/security/`.
- HTTP and socket maps: `Docs/detail_logique/network/01_http_api_map.md`, `Docs/detail_logique/network/02_socket_events_map.md`.
- Combination analysis report: `Docs/final_report/QUANTUM_COMBI.adoc`.